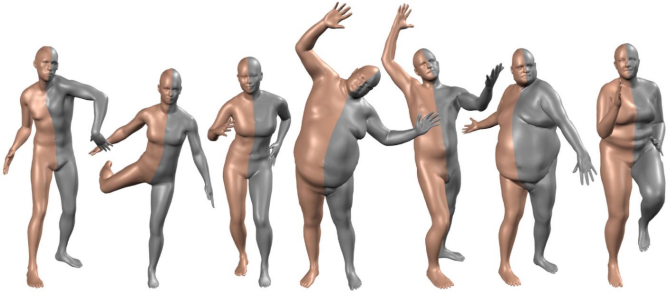
Foundations of 4D/6D Object Capture for Virtual Environments

Colors and Rendering

March 3, 2026

Last Lecture

→ **Human Models**

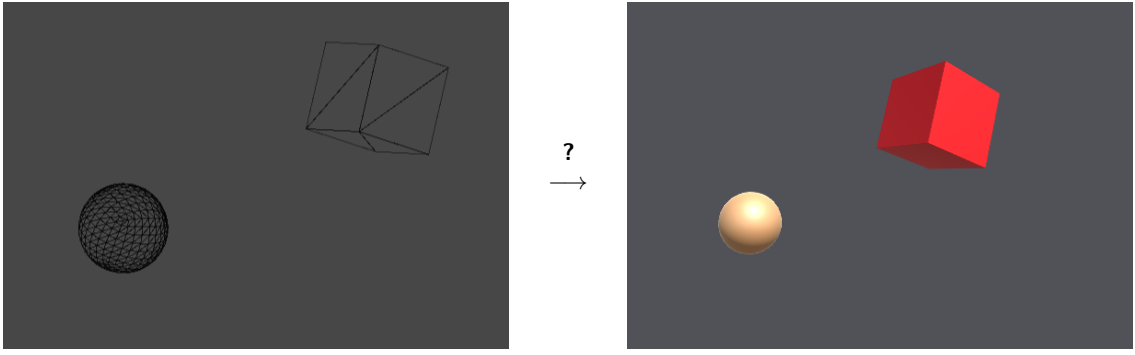


Today: Colors and Rendering

Rendering Task

Now, we have a baseline reconstruction method to create 3D models (meshes) from image data

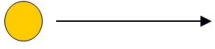
Question: How can we actually visualize them in some virtual environment?

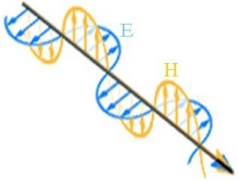


Light

Before talking about visualization, we need some fundamentals of the physics of light: **What is light?**

Light as a particle	Light as a wave
▶ Light travels along a straight line ▶ Light particles = photons	▶ Electro-magnetic wave ▶ Coupled electric (E) and magnetic (H) field

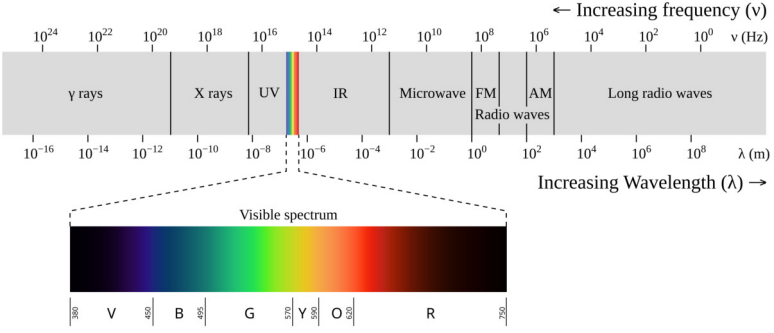




Wave - Particle Duality:

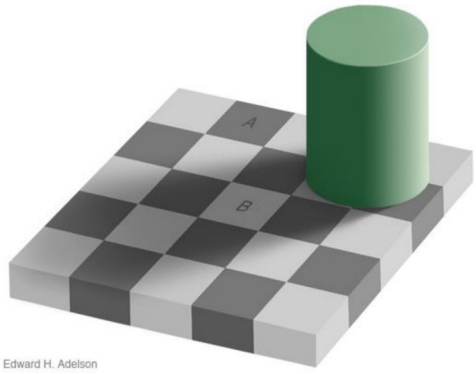
- Light has properties of a *wave*: frequency, phase, orientation
- Light has properties of a *particle*: "energy packets" = photons

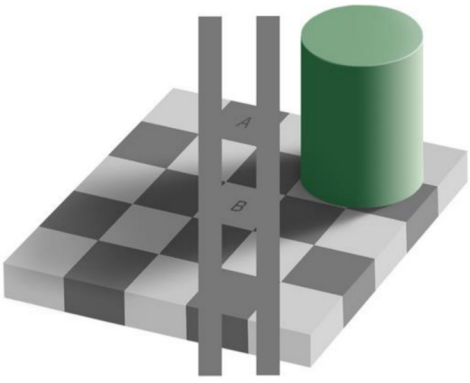
Visible spectrum:



Light

Observation: Colors are perceived subjectively → depends on context, person, . . .





Tristimulus Theory

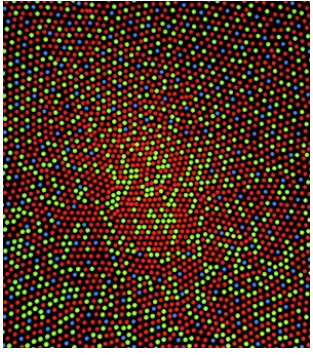
Tristimulus Theory: The perceived color of a spectral distribution can be described completely by three values

Reason: Spectral sensitivity of the three cone cell types (light receptors for color vision)

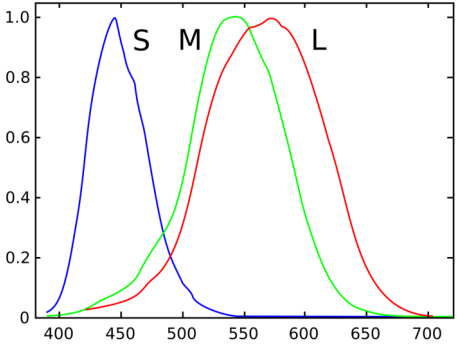
Distribution heavily varies between humans, e.g.:

L	M	S
60 %	30 %	10 %
50 %	45 %	5 %
...	...	...

→ Fewer cones (S) can be way more sensitive



Normalized Sensitivity:

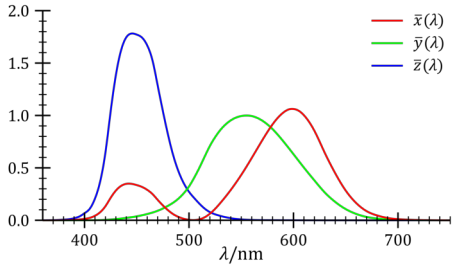


Tristimulus Theory

Idea: Perform color matching experiment based on 3 "primaries" → corresponds to cone types

Desired properties of primaries $\overline{x}, \overline{y}, \overline{z}$ **(CIE 1931 standard)**:

- Primaries should be non-negative for every λ
  → Additive model
- Second primary $\overline{y}$ should represent luminance
  → Effectively equal to "brightness"
- Equal-energy whitepoint should be at $(k, k, k)^T \in \mathbb{R}^3_{\geq 0}$
  → All wavelengths λ have same influence



The colors can be computed by projecting the spectrum S to the sensitivity curves:

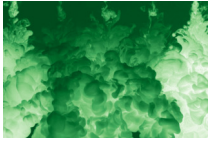
$$X = \int_\lambda S(\lambda)\,\overline{x}(\lambda)\,\mathrm{d}\lambda \quad , \quad Y = \int_\lambda S(\lambda)\,\overline{y}(\lambda)\,\mathrm{d}\lambda \quad , \quad Z = \int_\lambda S(\lambda)\,\overline{z}(\lambda)\,\mathrm{d}\lambda$$

Color Spaces

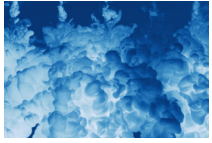
Many color spaces to represent an image:

- RGB
- HSV
- LAB
- etc. . . . (YUV, HSL, CMYK)

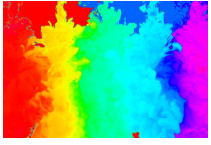
(a) RGB – R



(b) RGB – G

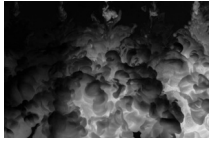


(c) RGB – B

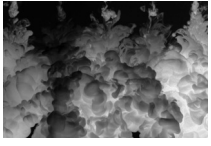


(d) HSV – H

(e) HSV – S



(f) HSV – V



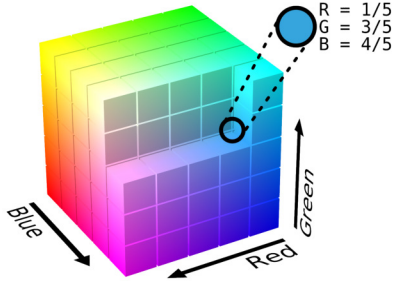
(g) LAB – L

(h) LAB – A

(i) LAB – B

Color Models: RGB

The RGB color space is defined by the three colors Red, Green, and Blue
Each color is represented as a vector in the color cube:



There is linear relationship between the XYZ and RGB color space via the transformation M:

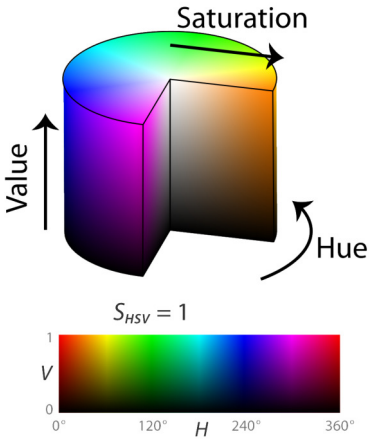
$$\begin{pmatrix} X \\ Y \\ Z \end{pmatrix} = M \cdot \begin{pmatrix} R \\ G \\ B \end{pmatrix}$$

Color Models: HSV

While the RGB color space is the most prominent color space, it is less suitable for picking and editing colors

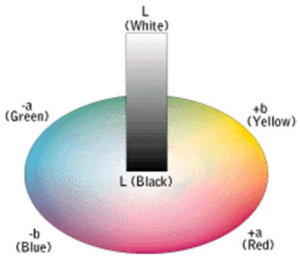
A more “intuitive” color space is the HSV color space:

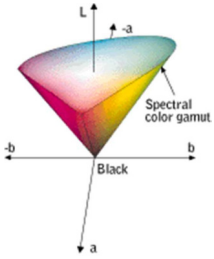
- Complementary colors are opposite to each other
- **H**: Hue, range: $[0°, 360°)$
- **S**: Saturation, range: $[0, 1]$
- **V**: Brightness, range: $[0, 1]$

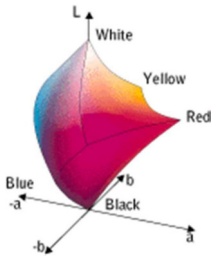


Color Models: LAB

Another color space is the LAB color space which is perceptually uniform color space
→ If one pair of colors has same Euclidean distance as another pair, these difference "appear similar"







- Brightness indicated vertically using a brightness scale L with values from 0 (black) to 100 (white)
- Saturation and hue defined by coordinates a and b, both of which can assume positive and negative values

Dynamic Range

So far, we only considered “color”, but not really “luminance” and dynamic range

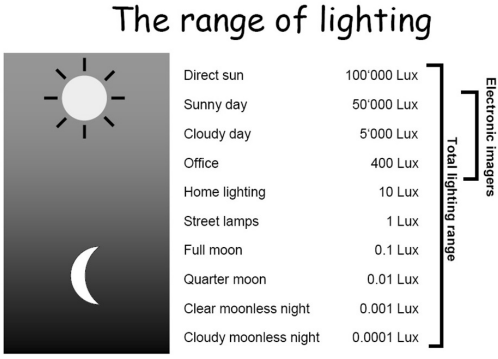
Problem:

- Perceivable dynamic range: $\approx 10^7$ values
- Typical cameras / displays: 256 values

Question: So how to map “High Dynamic Range (HDR)” data to “Low Dynamic Range (LDR)” displays?

$\rightarrow$ Tone Mapping

$$[0, L_{\max}] \mapsto [0, 1] \qquad L_{\max} \gg 1$$



Dynamic Range: Gamma Correction

Quantizing mapped LDR image to 8-bit (256 values):

(a) Original

(b) 8-bit Linear encoded

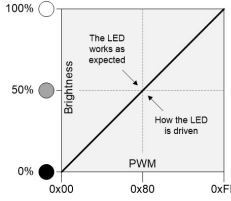
(c) 8-bit Gamma encoded

For a LDR image $I_{\text{in}} \in [0, 1]$:

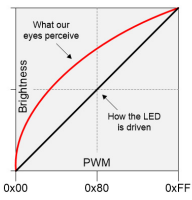
Linear Encoding: $I_{\text{out}} = \text{round}_{\text{8-bit}}(I_{\text{in}})$

Gamma Encoding: $I_{\text{out}} = \text{round}_{\text{8-bit}}(I_{\text{in}}^{1/\gamma})$ $\gamma = 2.2$

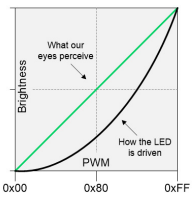
$$\text{round}_{\text{8-bit}}(I) = \frac{1}{255} \cdot \text{round}(255 \cdot I)$$



(a) What we expect to see



(b) What we actually see



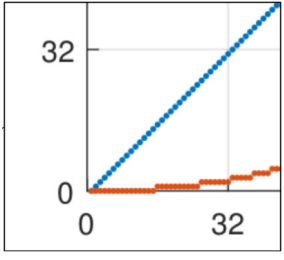
(c) Applying gamma correction

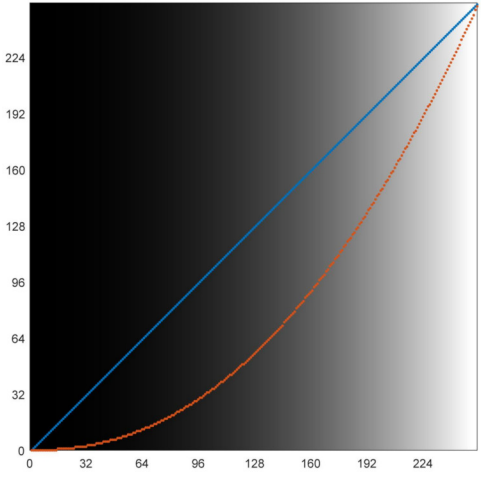
Dynamic Range: Gamma Correction

Gamma Encoding:

$$I_{\text{out}} = \text{round}_{\text{8-bit}}(I_{\text{in}}^{1/\gamma}) \quad \gamma = 2.2$$

Problem: Loss of precision near black at decoding



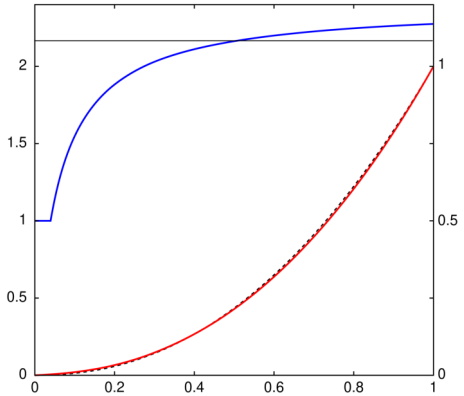


Dynamic Range: Gamma Correction

Solution: sRGB decoding curve

$$\mathrm{sRGB}^{-1}(I) = \begin{cases} \dfrac{I}{12.92} & \text{if } I \leq 0.04045 \\ \left(\dfrac{I + 0.055}{1.055}\right)^{2.4} & \text{otherwise} \end{cases}$$

- Linear ($\gamma = 1.0$) near black
- Slope gradually increases up to $\gamma \approx 2.3$
- Overall slope: $\gamma \sim 2.2$

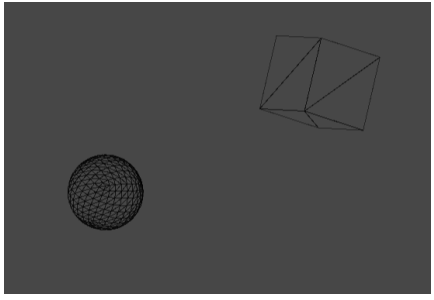


The sRGB encoding function is:

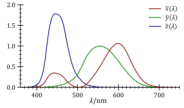
$$\mathrm{sRGB}(I) = \begin{cases} 12.92 \cdot I & \text{if } I \leq 0.0031308 \\ 1.055 \cdot I^{\frac{1}{2.4}} - 0.055 & \text{otherwise} \end{cases}$$

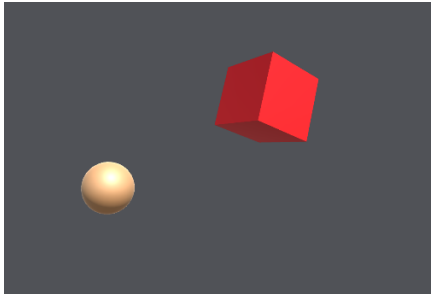
Back to the Rendering Task

Question: How does that knowledge about color and luminance help with visualizing virtual objects?



?





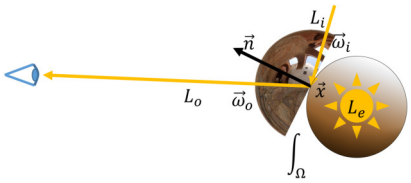
↓

Rendering Equation

Rendering Equation

Surface rendering can be fully described by the famous Rendering Equation[1][2]:

$$\boldsymbol{L}_o(\boldsymbol{x}, \boldsymbol{v}) = \underbrace{\boldsymbol{L}_e(\boldsymbol{x}, \boldsymbol{v})}_{\text{emitted radiance}} + \underbrace{\int_\Omega \underbrace{\boldsymbol{f}_{\text{BRDF}}(\boldsymbol{x}, \boldsymbol{l}, \boldsymbol{v})}_{\text{reflectance function}} \underbrace{\boldsymbol{L}_i(\boldsymbol{x}, \boldsymbol{l})}_{\substack{\text{incoming} \\ \text{radiance}}} \underbrace{\langle \boldsymbol{n} \,|\, \boldsymbol{l} \rangle}_{\substack{\text{cosine} \\ \text{attenuation}}} \, \mathrm{d}\boldsymbol{l}}_{\text{reflected radiance}}$$



Interpretation: The outgoing radiance ("amount of light") is equal to the directly emitted radiance plus all radiance that hits $\boldsymbol{x}$ from any direction $\boldsymbol{l}$ and is reflected to the direction $\boldsymbol{v}$

[1] J. T. Kajiya. "The rendering equation". In: *SIGGRAPH*. 1986.

[2] D. S. Immel et al. "A radiosity method for non-diffuse environments". In: *SIGGRAPH*. 1986.

BRDF

Strictly speaking, we only defined the Rendering Equation with *reflection* but without *transmission*:

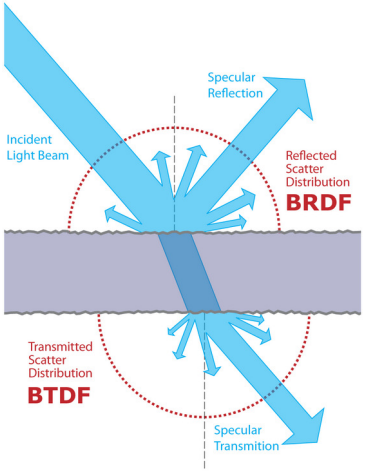
- **B**idirectional **R**eflectance **D**istribution **F**unction (BRDF)

$$f_{\text{BRDF}} : \Omega \times \Omega_r \to \mathbb{R}^n$$

- **B**idirectional **T**ransmittance **D**istribution **F**unction (BTDF)

$$f_{\text{BTDF}} : \Omega \times \Omega_t \to \mathbb{R}^n$$

Ω_r, Ω_t: hemispheres where light gets reflected/transmitted



Note:

→ If $f_{\text{BRDF}}, f_{\text{BTDF}}$ also depend on x, they are called spatially varying

→ For simplicity, we will typically only focus on BRDFs!

BRDF

For rendering, several properties of BRDFs are important:

- Helmholtz Reciprocity:

$$\boldsymbol{f}_{\text{BRDF}}(\boldsymbol{l}, \boldsymbol{v}) = \boldsymbol{f}_{\text{BRDF}}(\boldsymbol{v}, \boldsymbol{l})$$

  This means the incident and outgoing directions can be interchanged ($\rightarrow$ path tracing)

- Positivity:

$$\boldsymbol{f}_{\text{BRDF}}(\boldsymbol{l}, \boldsymbol{v}) \geq \mathbf{0}$$

- Energy Conservation:

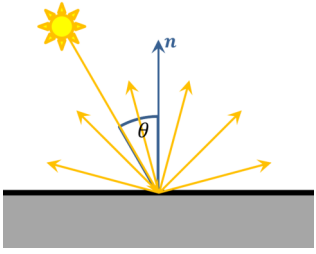
$$\boldsymbol{a} \leq \mathbf{1} \qquad \text{with albedo:} \quad \boldsymbol{a} := \int_{\Omega_r} \boldsymbol{f}_{\text{BRDF}}(\boldsymbol{l}, \boldsymbol{v}) \, \langle \, \boldsymbol{n} \, | \, \boldsymbol{v} \, \rangle \, \mathrm{d}\boldsymbol{v}$$

The following **simplified assumptions** were made:

- The wavelengths of light remain constant
- Light is directly reflected (phosphorescence is not modeled)
- Light transport between two objects occurs in a vacuum
- Atmospheric effects or reflection on hair are not accurately described

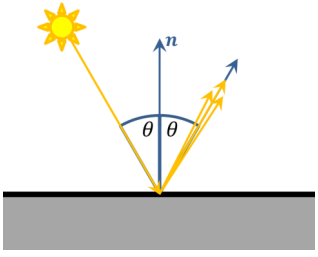
BRDF

Diffuse Reflectance



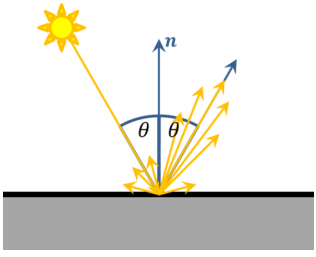
$f_{\text{BRDF}} = f_{\text{diffuse}}$

Specular Reflectance



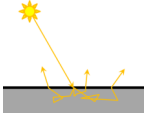
$f_{\text{BRDF}} = f_{\text{specular}}$

Composed Reflectance

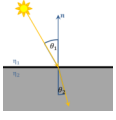


$f_{\text{BRDF}} = f_{\text{diffuse}} + f_{\text{specular}}$

There is also subsurface scattering and refraction, but this is not covered here



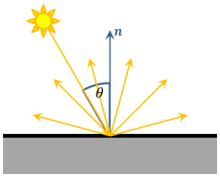
Subsurface Scattering

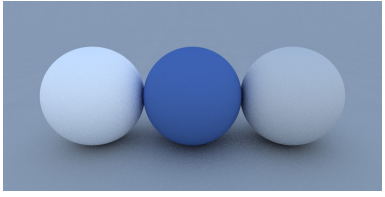


Refraction

Diffuse BRDF

First of all, consider diffuse – optically very rough – surfaces reflecting a portion of the incoming light with radiance uniformly distributed in all directions





Observations:

1. Diffuse BRDF is independent of outgoing direction $\boldsymbol{v}$:

$$\boldsymbol{f}_{\text{diffuse}}(\boldsymbol{l}, \boldsymbol{v}_1) = \boldsymbol{f}_{\text{diffuse}}(\boldsymbol{l}, \boldsymbol{v}_2) \quad \Rightarrow \quad \boldsymbol{f}_{\text{diffuse}}(\boldsymbol{l}, \boldsymbol{v}) = \underbrace{\boldsymbol{c}}_{\text{constant}}$$

2. Due to Helmholtz Reciprocity, also independent of incident direction $\boldsymbol{l}$:

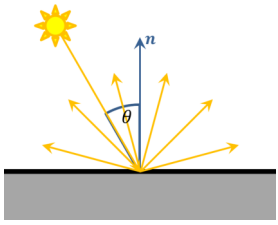
$$\boldsymbol{f}_{\text{diffuse}}(\boldsymbol{l}_1, \boldsymbol{v}) = \boldsymbol{f}_{\text{diffuse}}(\boldsymbol{l}_2, \boldsymbol{v})$$

Diffuse BRDF

We already know that the diffuse BRDF is constant: $\boldsymbol{f}_{\text{diffuse}}(\boldsymbol{l}, \boldsymbol{v}) = \boldsymbol{c}$

3. It must also be physically plausible:

$$\begin{aligned}\mathbf{1} &\geq \int_{\Omega_r} \boldsymbol{c} \cdot \langle\, \boldsymbol{n} \,|\, \boldsymbol{v} \,\rangle \,\mathrm{d}\boldsymbol{v} \\ &= \int_{\phi=0}^{2\pi} \int_{\theta=0}^{\pi/2} \boldsymbol{c} \cdot \cos(\theta) \sin(\theta) \,\mathrm{d}\theta \,\mathrm{d}\phi \\ &= \boldsymbol{c} \cdot \pi\end{aligned}$$



Definition (Diffuse BRDF[3])

The *Diffuse BRDF* or *Lambert-BRDF* is defined as

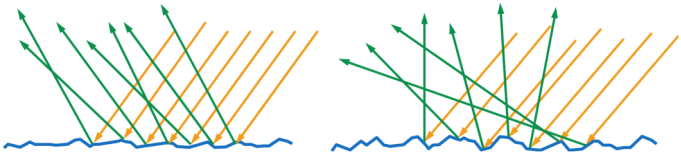
$$\boldsymbol{f}_{\text{diffuse}}(\boldsymbol{l}, \boldsymbol{v}) = \frac{\boldsymbol{a}}{\pi} \quad , \quad \boldsymbol{a} \leq \mathbf{1}$$

[3] J. Lambert and E. Anding. *Photometria, sive De mensura et gradibus luminis, colorum et umbrae (1760)*. Ostwalds Klassiker der exakten Wissenschaften. W. Engelmann, 1892.

Specular BRDF

For specular reflections:

Model the surface as a collection of infinitely small mirrors
$\rightarrow$ microfacets



Definition (Microfacet BRDF[4])

The *Microfacet BRDF* is defined as

$$\boldsymbol{f}_{\text{specular}}(\boldsymbol{l}, \boldsymbol{v}) = \frac{D(\boldsymbol{h}) \cdot G(\boldsymbol{l}, \boldsymbol{v}, \boldsymbol{h}) \cdot \boldsymbol{F}(\boldsymbol{l}, \boldsymbol{h})}{4 \langle \boldsymbol{n} \,|\, \boldsymbol{l} \rangle \langle \boldsymbol{n} \,|\, \boldsymbol{v} \rangle}$$

- D: Proportion of microfacets reflecting light
- G: Proportion of microfacets blocking each other
- $\boldsymbol{F}$: Reflectivity of microfacet

[4] R. L. Cook and K. E. Torrance. "A reflectance model for computer graphics". In: *ACM Transactions on Graphics (TOG)* 1.1 (1982).

Specular BRDF: D – Normal Distribution Function

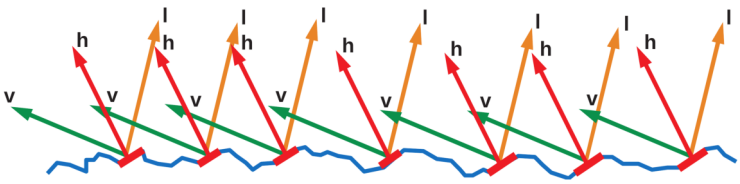
Trowbridge-Reitz (GGX)[5][6]:

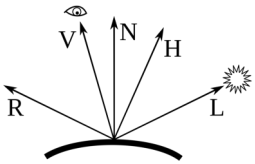
$$D(\boldsymbol{h}) = \frac{\alpha^2}{\pi \left(\langle \boldsymbol{n} \,|\, \boldsymbol{h} \rangle^2 \left(\alpha^2 - 1 \right) + 1 \right)^2}$$

Halfway vector:

$$\boldsymbol{h} = \frac{\boldsymbol{l} + \boldsymbol{v}}{\|\boldsymbol{l} + \boldsymbol{v}\|}$$

- Analytical approximation of how rough the surface is
- Only those microfacets whose normals $\boldsymbol{n}_{\text{micro}}$ align with $\boldsymbol{h}$ contribute





[5] T. Trowbridge and K. P. Reitz. "Average irregularity representation of a rough surface for ray reflection". In: *Journal of the optical society of America* 65.5 (1975).

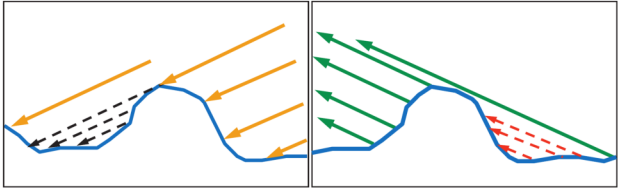
[6] B. Walter et al. "Microfacet Models for Refraction through Rough Surfaces". In: *Rendering techniques* 2007 (2007).

Specular BRDF: G – Geometric Attenuation Function

Smith Joint Masking-Shadowing Function for GGX[7]:

$$G(\boldsymbol{l}, \boldsymbol{v}, \boldsymbol{h}) = \frac{1}{1 + \Lambda(\boldsymbol{v}) + \Lambda(\boldsymbol{l})} \qquad \Lambda(\boldsymbol{v}) = \frac{-1 + \sqrt{\frac{\langle \boldsymbol{n} \mid \boldsymbol{v} \rangle^2 (1-\alpha^2)+\alpha^2}{\langle \boldsymbol{n} \mid \boldsymbol{v} \rangle^2}}}{2}$$

- Masking: Proportion of area occluded from light
- Shadowing: Proportion of area occluded from view



[7] E. Heitz. "Understanding the masking-shadowing function in microfacet-based BRDFs". In: *Journal of Computer Graphics Techniques* 3.2 (2014).

Specular BRDF: $\boldsymbol{F}$ – Fresnel Function

Schlick's approximation[8]:

$$\boldsymbol{F}(\boldsymbol{l}, \boldsymbol{h}) = \boldsymbol{F}_0 + (1 - \boldsymbol{F}_0)\,(1 - \langle\, \boldsymbol{v} \,|\, \boldsymbol{h} \,\rangle)^5 \qquad \boldsymbol{F}_0 = \frac{(\eta(\lambda) - 1)^2}{(\eta(\lambda) + 1)^2}$$

$\rightarrow \eta(\lambda)$ is the index of refraction and dependent on the wavelength λ

Note: In practice, only the "extreme" cases are typically considered:

Dielectric: $\boldsymbol{F}_0 = \mathbf{0.04}$

Conductor: $\boldsymbol{F}_0 = \boldsymbol{\kappa}_{\text{base}}$

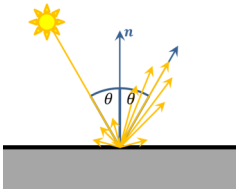
Material	F_0 (Linear)	F_0 (sRGB)	Color
Water	(0.02, 0.02, 0.02)	(0.15, 0.15, 0.15)	
Plastic / Glass (Low)	(0.03, 0.03, 0.03)	(0.21, 0.21, 0.21)	
Plastic High	(0.05, 0.05, 0.05)	(0.24, 0.24, 0.24)	
Glass (high) / Ruby	(0.08, 0.08, 0.08)	(0.31, 0.31, 0.31)	
Diamond	(0.17, 0.17, 0.17)	(0.45, 0.45, 0.45)	
Iron	(0.56, 0.57, 0.58)	(0.77, 0.78, 0.78)	
Copper	(0.95, 0.64, 0.54)	(0.98, 0.82, 0.76)	
Gold	(1.00, 0.71, 0.29)	(1.00, 0.86, 0.57)	
Aluminium	(0.91, 0.92, 0.92)	(0.96, 0.96, 0.97)	
Silver	(0.95, 0.93, 0.88)	(0.98, 0.97, 0.95)	

[8] C. Schlick. "An inexpensive BRDF model for physically-based rendering". In: *Computer Graphics Forum*. Vol. 13. 3. 1994.

Composed BRDF

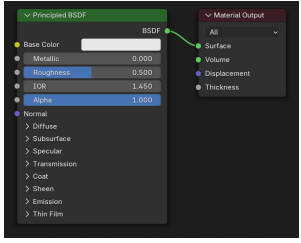
Most materials are not purely diffuse or purely specular

$$
\begin{aligned}
f_{\text{BRDF}}(\boldsymbol{l}, \boldsymbol{v}) &= f_{\text{diffuse}}(\boldsymbol{l}, \boldsymbol{v}) + f_{\text{specular}}(\boldsymbol{l}, \boldsymbol{v}) \\
&= \frac{\boldsymbol{a}}{\pi} + \frac{D(\boldsymbol{h}) \cdot G(\boldsymbol{l}, \boldsymbol{v}, \boldsymbol{h}) \cdot \boldsymbol{F}(\boldsymbol{l}, \boldsymbol{h})}{4 \langle \boldsymbol{n} \,|\, \boldsymbol{l} \rangle \langle \boldsymbol{n} \,|\, \boldsymbol{v} \rangle}
\end{aligned}
$$



Typically, this BRDF is parameterized by user-controllable parameters (Unreal Engine, Unity, Blender):

$$
\begin{aligned}
&\boldsymbol{\kappa}_{\text{base}} \in [0,1]^3 && \boldsymbol{a} = (1 - \text{metallic}) \cdot \boldsymbol{\kappa}_{\text{base}} \\
&\text{metallic} \in [0,1] \quad \Rightarrow && \boldsymbol{F}_0 = (1 - \text{metallic}) \cdot \mathbf{0.04} + \text{metallic} \cdot \boldsymbol{\kappa}_{\text{base}} \\
&\text{roughness} \in [0,1] && \alpha = \text{roughness}^2
\end{aligned}
$$



Question: How to solve the Rendering Equation?

Rasterization

- Extremely fast
- Limited accuracy

Ray Tracing

- More demanding
- Better reflections

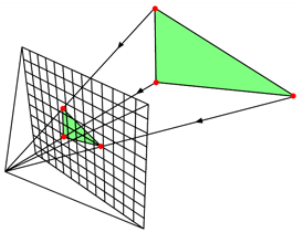
Path Tracing

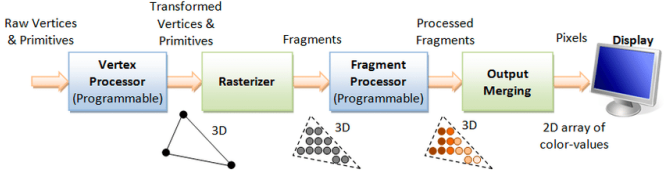
- Very expensive
- Global illumination

Rendering Techniques: Rasterization

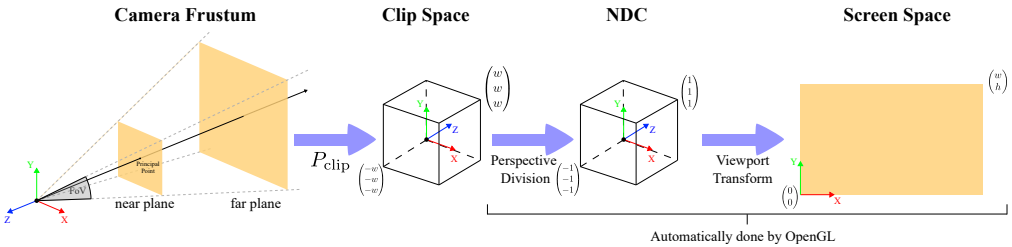
Idea: Project mesh geometry into image space and then shade it

1. Vertex Processor/Shader: Applies transformations to clip space
   → Extrinsics (object ↔ world ↔ camera) + Intrinsics ($\boldsymbol{P}_{\text{clip}}$)
2. Rasterizer: Interpolates projected data ($\boldsymbol{x}$, $\boldsymbol{n}$, $(u, v)^T$, ...) to pixels
   → Barycentric coordinates
3. Fragment Processor/Shader: Computes final color of pixels
   → Illumination with $\boldsymbol{f}_{\text{BRDF}}(\boldsymbol{x}, \boldsymbol{l}, \boldsymbol{v})$ and light sources
4. Output Merging: Collects colors of all fragments





Question: What is the clip space?



→ Intermediate space before perspective division and transformation to pixel coordinates

$$\boldsymbol{P}_{\text{clip}} = \begin{pmatrix} \frac{1}{\tan(\alpha_x/2)} & 0 & \sigma_x & 0 \\ 0 & \frac{1}{\tan(\alpha_y/2)} & \sigma_y & 0 \\ 0 & 0 & -\frac{f+n}{f-n} & -\frac{2\,f\,n}{f-n} \\ 0 & 0 & -1 & 0 \end{pmatrix}$$

- Field of view (FOV): α_x, α_y
- Offset to principal point: σ_x, σ_y (ideally 0)
- Near plane n, far plane f
- Geometry **outside** $[-w, w]$ is discarded → clipped

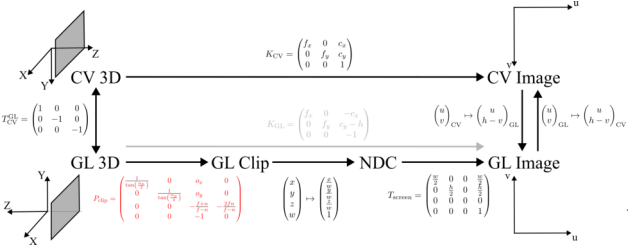
To render a reconstruction obtained from calibrated camera data, we need the respective clip space matrix:

OpenCV

$$\boldsymbol{K} = \begin{pmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix} \quad \rightarrow$$

OpenGL

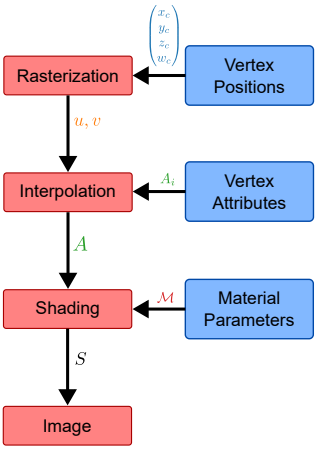
$$\boldsymbol{P}_{\text{clip}} = \begin{pmatrix} \frac{2f_x}{w} & 0 & 1-\frac{2c_x}{w} & 0 \\ 0 & \frac{2f_y}{h} & \frac{2c_y}{h}-1 & 0 \\ 0 & 0 & -\frac{f+n}{f-n} & -\frac{2\,f\,n}{f-n} \\ 0 & 0 & -1 & 0 \end{pmatrix}$$



	CV	GL
Output	Pixels	Clip coordinates
Z-axis	Forward	Backward
Y-axis	Down	Up

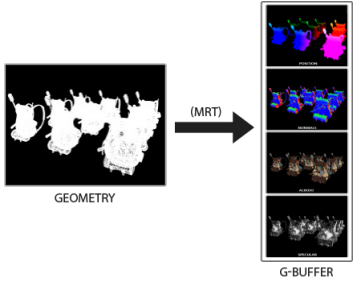
→ Memory layout of images also flipped in Y-axis!

In practice, rasterization is often performed via Deferred Shading, e.g. in many games



Idea: Decouple scene geometry from shading computation:

- First pass: Only vertex shader + collect foreground geometry in fragment shader



- Second pass: No (trivial) vertex shader + actual fragment shader
  → Performs (expensive) shading only on visible geometry

Rasterization primarily focuses on triangles and their projection, but the Rendering Equation actually considers ray $\leftrightarrow$ geometry intersections $\boldsymbol{x}$ the ray directions $\boldsymbol{l}, \boldsymbol{v}$:

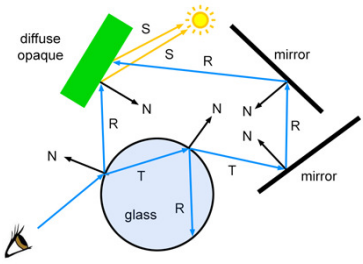
$$\boldsymbol{L}_o(\boldsymbol{x}, \boldsymbol{v}) = \boldsymbol{L}_e(\boldsymbol{x}, \boldsymbol{v}) + \int_{\Omega} \boldsymbol{f}_{\text{BRDF}}(\boldsymbol{x}, \boldsymbol{l}, \boldsymbol{v})\, \boldsymbol{L}_i(\boldsymbol{x}, \boldsymbol{l})\, \langle\, \boldsymbol{n} \,|\, \boldsymbol{l} \,\rangle \, \mathrm{d}\boldsymbol{l}$$

Idea: Some directions contribute more to the integral than others $\rightarrow$ Pick most important directions

- Visible light sources $\boldsymbol{L}_{e,\text{vls}}$ at $\boldsymbol{l}_{\text{vls}}$
- Ideal Reflection $\boldsymbol{L}_i$ at $\boldsymbol{l}_{\text{ir}}$
- Ideal Transmission $\boldsymbol{L}_i$ at $\boldsymbol{l}_{\text{it}}$

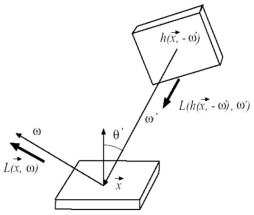
$$\begin{aligned}\boldsymbol{L}_o(\boldsymbol{x}, \boldsymbol{v}) \approx \boldsymbol{L}_e(\boldsymbol{x}, \boldsymbol{v}) &+ \sum_{\text{ls}} \boldsymbol{f}_{\text{BRDF}}(\boldsymbol{x}, \boldsymbol{l}_{\text{vls}}, \boldsymbol{v})\, \boldsymbol{L}_{e,\text{vls}}(\boldsymbol{x}, \boldsymbol{l}_{\text{vls}})\, \langle\, \boldsymbol{n} \,|\, \boldsymbol{l}_{\text{ls}} \,\rangle \\ &+ \boldsymbol{f}_{\text{BRDF}}(\boldsymbol{x}, \boldsymbol{l}_{\text{ir}}, \boldsymbol{v})\, \boldsymbol{L}_i(\boldsymbol{x}, \boldsymbol{l}_{\text{ir}})\, \langle\, \boldsymbol{n} \,|\, \boldsymbol{l}_{\text{ir}} \,\rangle \\ &+ \boldsymbol{f}_{\text{BTDF}}(\boldsymbol{x}, \boldsymbol{l}_{\text{it}}, \boldsymbol{v})\, \boldsymbol{L}_i(\boldsymbol{x}, \boldsymbol{l}_{\text{it}})\, \langle\, \boldsymbol{n} \,|\, \boldsymbol{l}_{\text{it}} \,\rangle\end{aligned}$$

$\rightarrow$ Like rasterization, this **does not model** Global Illumination effects



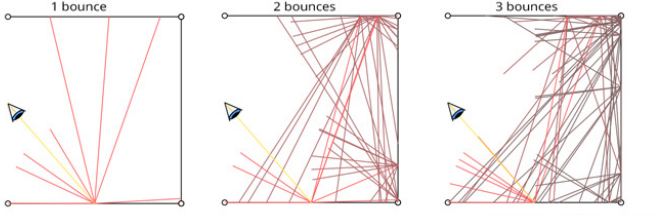
Observation: In Ray Tracing, we implicitly used a more general fact:

$$\boldsymbol{L}_i(\boldsymbol{x}, \boldsymbol{l}) = \boldsymbol{L}_o(\underbrace{\boldsymbol{h}(\boldsymbol{x}, \boldsymbol{l})}_{\substack{\text{hit point along} \\ \text{ray } \boldsymbol{x} + t \cdot \boldsymbol{l}}}, -\boldsymbol{l})$$



$\rightarrow$ The Rendering Equation is actually a **recursive** equation!

Question: How to solve an infinite series of increasingly higher-dimensional integrals, i.e. 2D + 4D + 6D + ...?



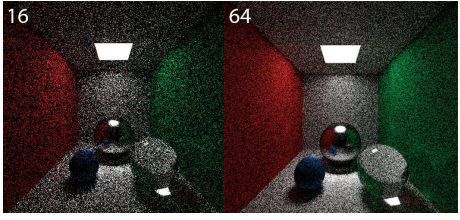
Rescue: Monte Carlo Estimator

$$I = \int_{\mathcal{V}} f(\boldsymbol{z})\,\mathrm{d}\boldsymbol{z} \qquad f\colon \mathcal{V} \subset \mathbb{R}^d \to \mathbb{R} \qquad \xRightarrow{\text{Random sampling with probability } p(\boldsymbol{z})} \qquad \hat{I} = \frac{1}{K}\sum_{k=1}^{K} \frac{f(\boldsymbol{z}_k)}{p(\boldsymbol{z}_k)}$$

- Estimate $\hat{I}$ is unbiased
- Convergence $\hat{I} \to I$ is independent of dimension $\mathbb{R}^d$: $2\times$ lower error $\hat{=}$ $4\times$ larger K

Applying this to the Rendering Equation leads to:

$$\hat{\boldsymbol{L}}_o(\boldsymbol{x}, \boldsymbol{v}) = \boldsymbol{L}_e(\boldsymbol{x}, \boldsymbol{v}) + \sum_{k=1}^{K} \frac{\boldsymbol{f}_{\text{BRDF}}(\boldsymbol{x}, \boldsymbol{l}_k, \boldsymbol{v})\, \boldsymbol{L}_i(\boldsymbol{x}, \boldsymbol{l}_k)\, \langle\, \boldsymbol{n} \,|\, \boldsymbol{l}_k \,\rangle}{p(\boldsymbol{l}_k)}$$

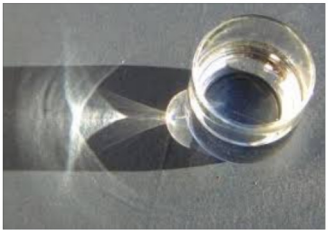


Example: Increasing samples per pixel

Rendering Techniques: Path Tracing

Results: Path Tracing can simulate all Global Illumination effects

Distortion effects from refractions



Caustics – concentration of light

→ THE gold standard, but real-time variants are still ongoing research

Next Lecture: Differentiable Rendering

Next Lecture

→ **Differentiable Rendering**

References I

[1] R. L. Cook and K. E. Torrance. "A reflectance model for computer graphics". In: *ACM Transactions on Graphics (TOG)* 1.1 (1982).

[2] E. Heitz. "Understanding the masking-shadowing function in microfacet-based BRDFs". In: *Journal of Computer Graphics Techniques* 3.2 (2014).

[3] D. S. Immel, M. F. Cohen, and D. P. Greenberg. "A radiosity method for non-diffuse environments". In: *SIGGRAPH*. 1986.

[4] J. T. Kajiya. "The rendering equation". In: *SIGGRAPH*. 1986.

[5] J. Lambert and E. Anding. *Photometria, sive De mensura et gradibus luminis, colorum et umbrae (1760)*. Ostwalds Klassiker der exakten Wissenschaften. W. Engelmann, 1892.

[6] C. Schlick. "An inexpensive BRDF model for physically-based rendering". In: *Computer Graphics Forum*. Vol. 13. 3. 1994.

[7] T. Trowbridge and K. P. Reitz. "Average irregularity representation of a rough surface for ray reflection". In: *Journal of the optical society of America* 65.5 (1975).

[8] B. Walter, S. R. Marschner, H. Li, and K. E. Torrance. "Microfacet Models for Refraction through Rough Surfaces". In: *Rendering techniques* 2007 (2007).